

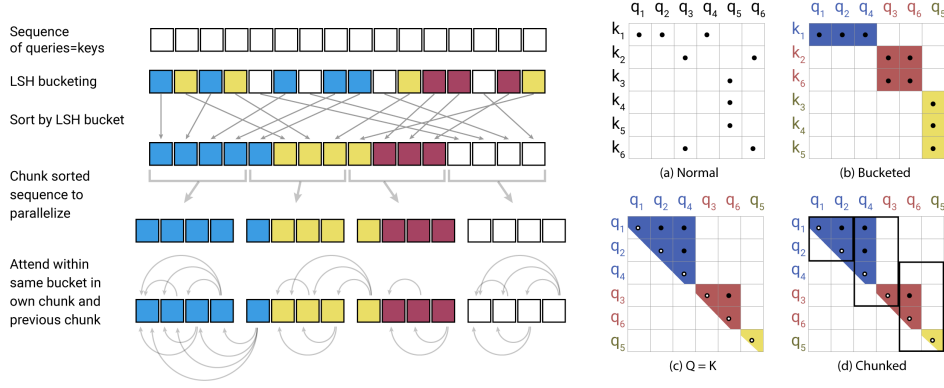


Figure 2: Simplified depiction of LSH Attention showing the hash-bucketing, sorting, and chunking steps and the resulting causal attentions. (a-d) Attention matrices for these varieties of attention.

Now we turn to LSH attention, which we can think of in terms of restricting the set $\mathcal{P}_i$ of target items a query position i can attend to, by only allowing attention within a single hash bucket.

$$\mathcal{P}_i = \{j : h(q_i) = h(k_j)\} \quad (4)$$

Figure 2(a-b) shows a schematic comparison of full-attention with a hashed variant. Part (a) depicts that the attention matrix for full attention is typically sparse, but the computation does not take advantage of this sparsity. In (b), the queries and keys have been sorted according to their hash bucket. Since similar items fall in the same bucket with high probability, the full attention pattern can be approximated by only allowing attention within each bucket.

Hash buckets in this formulation tend to be uneven in size, which makes it difficult to batch across buckets. Moreover, the number of queries and the number of keys within a bucket may be unequal – in fact, it is possible for a bucket to contain many queries but no keys. To alleviate these issues, we first ensure that $h(k_j) = h(q_j)$ by setting $k_j = \frac{q_j}{\|q_j\|}$. Next, we sort the queries by bucket number and, within each bucket, by sequence position; this defines a permutation where $i \mapsto s_i$ after sorting. In the sorted attention matrix, pairs from the same bucket will cluster near the diagonal (as depicted in Figure 2c). We can follow a batching approach where chunks of m consecutive queries (after sorting) attend to each other, and one chunk back (Figure 2d). Following our earlier notation, this corresponds to setting:

$$\tilde{\mathcal{P}}_i = \left\{ j : \left\lfloor \frac{s_i}{m} \right\rfloor - 1 \leq \left\lfloor \frac{s_j}{m} \right\rfloor \leq \left\lfloor \frac{s_i}{m} \right\rfloor \right\} \quad (5)$$

If $\max_i |\mathcal{P}_i| < m$, then $\mathcal{P}_i \subseteq \tilde{\mathcal{P}}_i$. In practice we set $m = \frac{2l}{n_{\text{buckets}}}$ (where l is the sequence length). The average bucket size is $\frac{l}{n_{\text{buckets}}}$, and we assume that the probability of a bucket growing to twice that size is sufficiently low. The overall process of LSH attention is summarized in Figure 2.

Multi-round LSH attention. With hashing, there is always a small probability that similar items nevertheless fall in different buckets. This probability can be reduced by doing multiple rounds of hashing with n_{rounds} distinct hash functions $\{h^{(1)}, h^{(2)}, \dots\}$, such that:

$$\mathcal{P}_i = \bigcup_{r=1}^{n_{\text{rounds}}} \mathcal{P}_i^{(r)} \quad \text{where } \mathcal{P}_i^{(r)} = \left\{ j : h^{(r)}(q_i) = h^{(r)}(q_j) \right\} \quad (6)$$

The multi-round case essentially involves performing LSH attention n_{rounds} times in parallel; the details of the procedure are described in in Appendix A.

Causal masking for shared-QK attention. In a Transformer decoder, masking (denoted by $m(j, \mathcal{P}_i)$ in Equation 3) is used to prevent positions from attending into the future. To implement masking in LSH attention, we associate every query/key vector with a position index, re-order the position indices using the same permutations used to sort the query/key vectors, and then use a comparison operation to compute the mask.

Table 1: Memory and time complexity of attention variants. We write l for length, b for batch size, n_h for the number of heads, n_c for the number of LSH chunks, n_r for the number of hash repetitions.

Attention Type	Memory Complexity	Time Complexity
Scaled Dot-Product	$\max(bn_hld_k, bn_hl^2)$	$\max(bn_hld_k, bn_hl^2)$
Memory-Efficient	$\max(bn_hld_k, bn_hl^2)$	$\max(bn_hld_k, bn_hl^2)$
LSH Attention	$\max(bn_hld_k, bn_hln_r(4l/n_c)^2)$	$\max(bn_hld_k, bn_hn_rl(4l/n_c)^2)$

Table 2: Accuracies on the duplication task of a 1-layer Transformer model with full attention and with locality-sensitive hashing attention using different number of parallel hashes.

Train \ Eval	Full Attention	LSH-8	LSH-4	LSH-2	LSH-1
Full Attention	100%	94.8%	92.5%	76.9%	52.5%
LSH-4	0.8%	100%	99.9%	99.4%	91.9%
LSH-2	0.8%	100%	99.9%	98.1%	86.8%
LSH-1	0.8%	99.9%	99.6%	94.8%	77.9%

While attention to the future is not allowed, typical implementations of the Transformer *do* allow a position to attend to *itself*. Such behavior is undesirable in a shared-QK formulation because the dot-product of a query vector with itself will almost always be greater than the dot product of a query vector with a vector at another position. We therefore modify the masking to forbid a token from attending to itself, except in situations where a token has no other valid attention targets (e.g. the first token in a sequence).

2.1 ANALYSIS ON A SYNTHETIC TASK

To verify the performance of LSH attention and study its behavior, we start with the following synthetic task: duplicate a sequence of symbols. In this task, each training and testing example has the form $0w0w$ where $w \in \{1, \dots, N\}^*$ is a sequence of symbols ranging from 1 to N (we use $N = 127$ in our experiments). An example with the word w of length 3 is given below.

Example:	0	19	113	72	0	19	113	72
-----------------	---	----	-----	----	---	----	-----	----

To study LSH attention, we train a language model on examples of the above form where each w is of length 511 (so the whole input $0w0w$ is of length 1024). As this is a language modeling task, we always predict the next symbol given all the previous ones, but we mask the loss and accuracy to only consider positions in the second half of the input, i.e., those that can actually be predicted.

The above task can be solved perfectly (to accuracy 100% and loss 0) by a 1-layer Transformer model. Note though, that it requires non-local attention lookups, so it cannot be solved by any model relying on sparse attention with a limited span. To make it easy and fast to train but similar to models used in NLP, we use a 1-layer Transformer with $d_{model} = d_{ff} = 256$ and 4 heads. We train it for 150K steps in 4 different settings: with full attention, LSH attention with $n_{rounds} = 1$, $n_{rounds} = 2$ and $n_{rounds} = 4$.

From the results summarized in Table 2 we see that a model trained with full attention can be immediately used with LSH attention, but at some loss of accuracy. When trained from scratch with LSH attention, the model trained with 4 hashes achieves almost perfect accuracy as well. Interestingly, the accuracy becomes perfect when evaluated with 8 hashes. It goes down when evaluated with 2 or 1 hashes. Models trained with less hashes show worse results but even the model trained with just 1 hash performs almost perfectly when evaluated with 8 hashes.

3 REVERSIBLE TRANSFORMER

As the above section shows, the complexity of attention can be reduced from square in length to linear, provided an approximation is acceptable. But it is clear from Table 1 that each field starts with a $b \cdot n_h \cdot l$ term: the $b \cdot n_h \cdot l \cdot d_k$, or alternatively $b \cdot l \cdot d_{model}$ cost cannot be avoided. Indeed, the activations before each layer are already of the size $b \cdot l \cdot d_{model}$, so the memory use of the whole model with n_l layers is at least $b \cdot l \cdot d_{model} \cdot n_l$. Even worse: inside the feed-forward layers of Transformer this goes up to $b \cdot l \cdot d_{ff} \cdot n_l$. In a big Transformer it is usual to set $d_{ff} = 4K$ and $n_l = 16$ so with $l = 64K$ this again would use an impractical $16GB$ of memory

In this section, we show how to reduce this cost by first dealing with the n_l part of the term using reversible layers and then showing how chunking can allow us to handle the d_{ff} problem. The effects of each of these approaches on memory and time complexity are summarized in Table 3.

RevNets. Reversible residual networks were introduced by Gomez et al. (2017) where it was shown that they can replace ResNets for image classification. The main idea is to allow the activations at any given layer to be recovered from the activations at the following layer, using only the model parameters. Rather than having to checkpoint intermediate values for use in the backward pass, layers can be reversed one-by-one as back-propagation proceeds from the output of the network to its input. Whereas a normal residual layer performs a function $x \mapsto y$ that operates on a single input and produces a single output and has the form $y = x + F(x)$, a reversible layer works on pairs of inputs/outputs: $(x_1, x_2) \mapsto (y_1, y_2)$, and follows the equations:

$$y_1 = x_1 + F(x_2) \quad y_2 = x_2 + G(y_1) \quad (7)$$

A layer can be reversed by subtracting (rather than adding) the residuals:

$$x_2 = y_2 - G(y_1) \quad x_1 = y_1 - F(x_2) \quad (8)$$

Reversible Transformer. We apply the RevNet idea to the Transformer by combining the attention and feed-forward layers inside the revnet block. In the notation above, F becomes an attention layer while G becomes the feed-forward layer. Note that Layer Normalization (Ba et al., 2016) is moved inside the residual blocks.

$$Y_1 = X_1 + \text{Attention}(X_2) \quad Y_2 = X_2 + \text{FeedForward}(Y_1) \quad (9)$$

The reversible Transformer does not need to store activations in each layer and so gets rid of the n_l term. In Section 5 we show that it performs the same as the normal Transformer when using the same number of parameters; we achieve this by having both x_1 and x_2 have size d_{model} .

Chunking. While reversibility covers the n_l term, the thicker layers can still use a lot of memory. The feed-forward layer in particular can use intermediate vectors of dimensionality $d_{ff} = 4K$ or higher. However, computations in feed-forward layers are completely independent across positions in a sequence, so the computation can be split into c chunks:

$$Y_2 = [Y_2^{(1)}; \dots; Y_2^{(c)}] = [X_2^{(1)} + \text{FeedForward}(Y_1^{(1)}); \dots; X_2^{(c)} + \text{FeedForward}(Y_1^{(c)})] \quad (10)$$

This layer is typically batched by performing operations for all positions in parallel, but operating on one chunk at a time can reduce memory. The reverse computation in (8) and the backward pass are also chunked. In addition to the feed-forward layers, for models with large vocabulary (more than d_{model} word types) we also chunk the log-probabilities at the output and calculate the loss for sections of the sequence at a time.

Chunking, large batches and parameter reuse. With chunking and reversible layers the memory we use for activations in the whole network is independent of the number of layers. The same is not true for parameters though as their number grows with the number of layers. This problem is remedied though because we can swap layer parameters to and from CPU memory when this layer is not computing. In a standard Transformer this would be inefficient because memory transfer to CPU is slow. The batch size multiplied by length in Reformer is much larger though and therefore the amount of compute done with the parameters amortizes the cost of their transfer.